



Laboratório 06: Herança, classe abstrata, interfaces e polimorfismo

20/05/2026

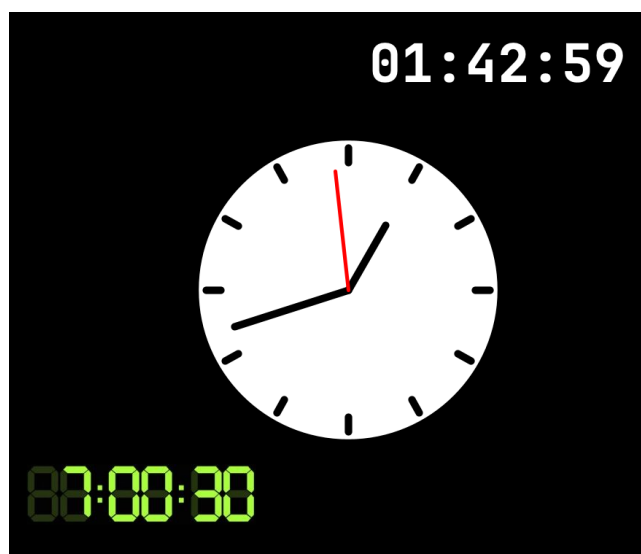
1 Relógios e cronômetros

Um relógio é um dispositivo que mede e exibe o tempo. Neste laboratório, você vai trabalhar com duas representações: **relógio digital** (hora em formato numérico) e **relógio analógico** (hora indicada por ponteiros). Para relógios digitais, você também vai implementar o comportamento de **cronômetro**, que pode ser:

- progressivo: inicia em 00:00:00 e avança a cada segundo
- regressivo: inicia em um valor definido pelo usuário e reduz até 00:00:00

Na Figura 1 são apresentados três exemplos de relógios: um relógio digital composto por 6 *displays* de 7 segmentos, um relógio analógico com ponteiros para horas, minutos e segundos, e um relógio digital representado por um texto simples.

Figura 1: Exemplo de relógios analógicos e digitais.



Neste laboratório, você deverá modelar um conjunto de classes para representar:

- um relógio digital com 6 *displays* de 7 segmentos
- um relógio analógico
- um relógio digital representado por texto simples

Reutilize o código do laboratório anterior do relógio com 6 *displays* e faça as adaptações necessárias. A área de desenho deve ser atualizada a cada segundo para refletir o avanço do tempo. Além disso, você deve atender aos seguintes requisitos:

- Deve ser possível instanciar relógios digitais e analógicos com a hora atual do sistema ou com uma hora definida manualmente. Os relógios digitais devem permitir três tamanhos (pequeno, médio e grande), enquanto os analógicos terão tamanho único. O tamanho da fonte do relógio digital representado por texto simples poderá ser informado pelo usuário, mas deve ser limitado a um intervalo de valores (por exemplo, entre 10 e 40);
- Somente os relógios digitais podem atuar como cronômetros progressivos, onde a hora inicial é 00:00:00 e o tempo avança a cada segundo, ou como cronômetros regressivos, onde a hora inicial é configurada pelo usuário e o tempo conta regressivamente até chegar a 00:00:00. Assim, ao criar um relógio digital deverá ser possível indicar se ele atuará como um relógio comum, um cronômetro progressivo ou um cronômetro regressivo;
- Faça uso de herança, classes abstratas e interfaces para definir comportamentos comuns entre os relógios, como a capacidade de avançar o tempo, desenhar na tela, configurar a hora, etc;
- Na classe com o método `public static void main(String args[])`, por exemplo *App.java*, deve-se criar uma área de desenho (um objeto da classe *Draw*). Esta classe será a responsável por periodicamente (a cada 1 segundo) limpar a área de desenho (*canvas*) e invocar os métodos dos relógios para que eles atualizem seus contadores e se desenhem novamente na área de desenho. Crie uma coleção de relógios (por exemplo, um `ArrayList<Relogio>`) e adicione diferentes tipos de relógios a essa coleção para que eles sejam atualizados e desenhados na tela a cada segundo. Lembre-se de utilizar polimorfismo para tratar os objetos da coleção de forma genérica, sem precisar conhecer o tipo específico de cada relógio.

2 Material de apoio com a biblioteca algs4

2.1 Importando a biblioteca algs4 em um projeto Java com gradle

1. Baixe o arquivo `algs4.jar` e coloque-o na pasta `libs` do projeto gradle. Se a pasta `libs` não existir, crie-a na raiz do projeto ou em `app/libs`
2. Adicione a dependência abaixo no arquivo `build.gradle.kts` para importar a biblioteca:

```
dependencies {
    implementation(files("libs/algs4.jar"))
}
```

3. Após adicionar a dependência, sincronize o projeto para que o gradle reconheça a biblioteca
4. Se estiver usando git, verifique se os arquivos JAR em `app/libs` não estão ignorados no `.gitignore`. Para isso, adicione:

```
# Não ignore os arquivos JAR contidos dentro do diretório app/libs
!**/app/libs/*.jar
```

2.2 Exemplo para carregar fonte personalizada em Java

Na Listagem 1 é apresentado um exemplo de código Java para carregar uma fonte personalizada a partir de um arquivo TTF. Você pode baixar fontes gratuitas em sites como <https://fonts.google.com/> e colocar o arquivo TTF na pasta de recursos do seu projeto (por exemplo, `src/main/resources`).

Em <https://www.dafont.com/seven-segment.font> você pode encontrar uma fonte que simula o estilo de um display de 7 segmentos. Em <https://www.jetbrains.com/pt-br/lp/mono/> há uma fonte monoespaçada que foi a usada no exemplo da Listagem 1.

Listagem 1: Exemplo com fonte personalizada

```
package poo;

import edu.princeton.cs.algs4.Draw;

import java.awt.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

/**
 * Exemplo de uso de fontes personalizadas em Java utilizando a classe Font. Neste exemplo, carregamos uma fonte TrueType
 * a partir de um arquivo e a utilizamos para desenhar a hora atual na tela. Certifique-se de ter o arquivo
 * "JetBrainsMono-Regular.ttf" disponível no diretório de recursos (src/main/resources) do projeto
 *
 * A fonte JetBrains Mono pode ser baixada gratuitamente em: https://www.jetbrains.com/pt-br/lp/mono/
 */
public class App {
    static void main(String[] args) throws Exception {
        Draw draw = new Draw();
        draw.enableDoubleBuffering();
        // Para finalizar a aplicação ao clicar no fechar da janela
        draw.setDefaultCloseOperation(3);

        // Carrega a fonte personalizada a partir do arquivo em src/main/resources
        var arquivo = App.class.getClassLoader().getResourceAsStream("JetBrainsMono-Regular.ttf");

        Font fontePersonalizada = Font.createFont(Font.TRUETYPE_FONT, arquivo);
        // Define o tamanho da fonte para 36 pontos
        fontePersonalizada = fontePersonalizada.deriveFont(36f);

        draw.setFont(fontePersonalizada);

        // Formata a hora atual para o formato "hh:mm:ss" (12 horas) ou "HH:mm:ss" (24 horas)
        DateTimeFormatter dateTimeFormatter = DateTimeFormatter.ofPattern("hh:mm:ss");

        draw.text(.5, .5, LocalDateTime.now().format(dateTimeFormatter));
        draw.show();
    }
}
```

2.3 Exemplo de relógio analógico com ponteiros

Na Listagem 2, há um exemplo de desenho de relógio analógico com algs4. A tela é atualizada a cada segundo para exibir a hora atual do sistema.

Listagem 2: Relógio analógico com algs4

```
package poo;

import edu.princeton.cs.algs4.Draw;
import java.time.LocalDateTime;
import java.util.concurrent.TimeUnit;

public class App {
    static void main(String[] args) throws InterruptedException {
        Draw draw = new Draw();
        draw.enableDoubleBuffering();
        // Para finalizar a aplicação ao clicar no fechar da janela
        draw.setDefaultCloseOperation(3);

        // Cores
        var corFundo = Draw.LIGHT_GRAY;
        var corRelogio = Draw.WHITE;
        var corTracos = Draw.BLACK;
        var corSegundos = Draw.RED;
```

```

// Centro do relógio
double centroX = 0.5;
double centroY = 0.5;
// Raio do círculo do relógio
double raioRelogio = 0.2;

// Proporções relativas ao raio do relógio
double comprimentoPonteiroSegundo = raioRelogio * 0.8;
double comprimentoPonteiroHora = raioRelogio * 0.5;
double raioInicioTraco = raioRelogio * 0.85;
double raioFimTraco = raioRelogio * 0.95;
double espessuraTraco = raioRelogio * 0.05;
double espessuraSegundo = raioRelogio * 0.025;

// Quantidade de traços (horas)
int tracosDasHoras = 12;
double anguloEntreTracos = 30.0; // graus, pois 360° / 12 = 30°

while(true) {
    draw.clear(corFundo);
    draw.setPenColor(corRelogio);
    draw.filledCircle(centroX, centroY, raioRelogio);

    draw.setPenColor(corTracos);
    draw.setPenRadius(espessuraTraco);

    for (int traco = 0; traco < tracosDasHoras; traco++) {
        double angulo = Math.toRadians(anguloEntreTracos * traco);
        //https://brasilescola.uol.com.br/matematica/simetria-no-circulo-trigonometrico.htm
        draw.line(
            centroX + raioFimTraco * Math.sin(angulo), centroY + raioFimTraco * Math.cos(angulo),
            centroX + raioInicioTraco * Math.sin(angulo), centroY + raioInicioTraco * Math.cos(angulo)
        );
    }

    // Obtendo a hora atual do sistema
    LocalTime agora = LocalTime.now();
    int hora = agora.getHour();
    int minuto = agora.getMinute();
    int segundo = agora.getSecond();

    double anguloHora = Math.toRadians(anguloEntreTracos * hora); // 360° / 12 = 30° por hora
    double anguloMinuto = Math.toRadians(6 * minuto); // 360° / 60 = 6° por minuto
    double anguloSegundo = Math.toRadians(6 * segundo); // 360° / 60 = 6° por segundo

    // Ponteiro das horas
    draw.line(centroX, centroY, centroX + comprimentoPonteiroHora * Math.sin(anguloHora), centroY +
comprimentoPonteiroHora * Math.cos(anguloHora));
    // Ponteiro dos minutos
    draw.line(centroX, centroY, centroX + comprimentoPonteiroSegundo * Math.sin(anguloMinuto), centroY +
comprimentoPonteiroSegundo * Math.cos(anguloMinuto));

    // Ponteiro dos segundos
    draw.setPenColor(corSegundos);
    draw.setPenRadius(espessuraSegundo);
    draw.line(centroX, centroY, centroX + comprimentoPonteiroSegundo * Math.sin(anguloSegundo), centroY +
comprimentoPonteiroSegundo * Math.cos(anguloSegundo));
    draw.show();
    TimeUnit.SECONDS.sleep(1);
}
}
}

```

2.4 Contador de 0 até 10 usando a biblioteca algs4

Na Listagem 3 é apresentado um exemplo de rotina periódica para limpar a área de desenho (*canvas*) e desenhar o valor de um contador. O método `TimeUnit.SECONDS.sleep(1)` é usado para pausar o aplicativo por 1 segundo em cada iteração do laço, enquanto o contador evolui de 0 até 10.

Listagem 3: Contador com algs4

```
package poo;
import edu.princeton.cs.algs4.Draw;
import java.util.concurrent.TimeUnit;

public class App{
    // Não iremos tratar as possíveis exceções de execução do programa, por isso o throws Exception
    public static void main(String[] args) throws Exception{
        Draw desenho = new Draw();
        desenho.setCanvasSize(800, 800);
        desenho.setXscale(0, 800);
        desenho.setYscale(0, 800);

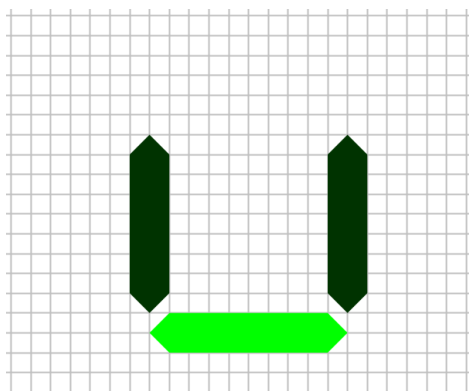
        // Toda ação de desenho acontecerá em um buffer secundário
        // e esse buffer só será visto depois que for invocado o método show()
        desenho.enableDoubleBuffering();

        for (int i = 1; i <= 10; i++) {
            // limpando a área de desenho
            desenho.clear();
            // escrevendo o valor de i na coordenada (500,500)
            desenho.text(500, 500, ""+ i);
            // Trocando o buffer para exibir o que foi escrito
            desenho.show();
            // Dormindo por 1 segundo
            TimeUnit.SECONDS.sleep(1);
        }
    } // fim do main
} // fim da classe
```

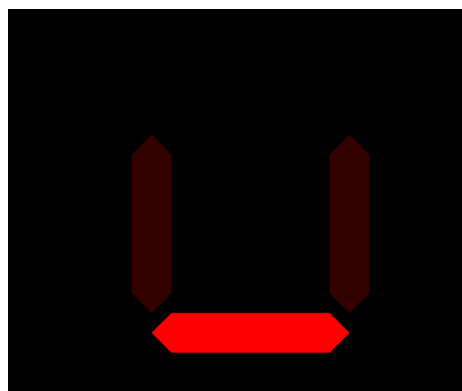
3 Desenhando polígonos preenchidos com a biblioteca algs4

Cada segmento do *display* de 7 segmentos pode ser desenhado como um polígono, assim você poderá usar o método `filledPolygon(double[] x, double[] y)` da classe `Draw`. Na Figura 2 são apresentadas capturas de tela obtidas a partir do código Java apresentado na Listagem 4.

Figura 2: Captura de telas com desenho de três polígonos



(a) Fundo branco com grade para facilitar visualização durante o desenvolvimento



(b) Fundo preto sem grade, ideal para entrega do projeto

Listagem 4: Desenhando polígonos com biblioteca algs4

```
package poo;
import edu.princeton.cs.algs4.Draw;
import java.awt.Color;

public class Poligonos{
    public static void main(String[] args){
        Draw desenho = new Draw();
        int dimensao = 800;
        desenho.setCanvasSize(dimensao, dimensao);
        desenho.setXscale(0, dimensao);
        desenho.setYscale(0, dimensao);

        double fator = 200;
        double fatorCor = 0.2;
        Color clara = Draw.GREEN;
        Color escura = new Color((int)(clara.getRed()*fatorCor),
                                   (int)(clara.getGreen()*fatorCor),
                                   (int)(clara.getBlue()*fatorCor));

        desenho.enableDoubleBuffering();
        desenho.clear(Draw.WHITE);
        // Desenhando grade quadriculada
        int grade = (int) fator/10;
        desenho.setPenColor(Draw.LIGHT_GRAY);
        for (int i = 0; i <= dimensao; i += grade) {
            desenho.line(i, 0, i, dimensao); // linha vertical
            desenho.line(0, i, dimensao, i); // linha horizontal
        }
        double xInicial = 300;
        double yInicial = 400;

        // Montando vetores com os pontos em X e em Y para desenhar um segmento horizontal
        yInicial = 180;
        double[] xHorizontal = {0.1*fator+xInicial, 0.2*fator+xInicial, 1.0*fator+xInicial, 1.1*fator+xInicial, 1.0*fator+xInicial, 0.2*fator+xInicial};
        double[] yHorizontal = {0.2*fator+yInicial, 0.3*fator+yInicial, 0.3*fator+yInicial, 0.2*fator+yInicial, 0.1*fator+yInicial, 0.1*fator+yInicial};

        // Desenhando o segmento horizontal
        desenho.setPenColor(clara);
        desenho.filledPolygon(xHorizontal, yHorizontal);

        // Montando vetores com os pontos em X e em Y para desenhar um segmento vertical
        yInicial = 200;
        double[] xVertical = {0.1*fator+xInicial, 0.2*fator+xInicial, 0.2*fator+xInicial, 0.1*fator+xInicial, 0*fator+xInicial, 0*fator+xInicial};
        double[] yVertical = {0.2*fator+yInicial, 0.3*fator+yInicial, 1.0*fator+yInicial, 1.1*fator+yInicial, 1.0*fator+yInicial, 0.3*fator+yInicial};

        // Desenhando o segmento vertical
        desenho.setPenColor(escura);
        desenho.filledPolygon(xVertical, yVertical);

        // Desenhando outro segmento vertical, porém com x deslocado em 'fator' pixels
        for (int i = 0; i < xVertical.length; i++) {
            xVertical[i]+=fator;
        }
        desenho.filledPolygon(xVertical, yVertical);
        // Trocando o buffer para exibir o que foi desenhado
        desenho.show();
    } // fim do main
} // fim da classe
```